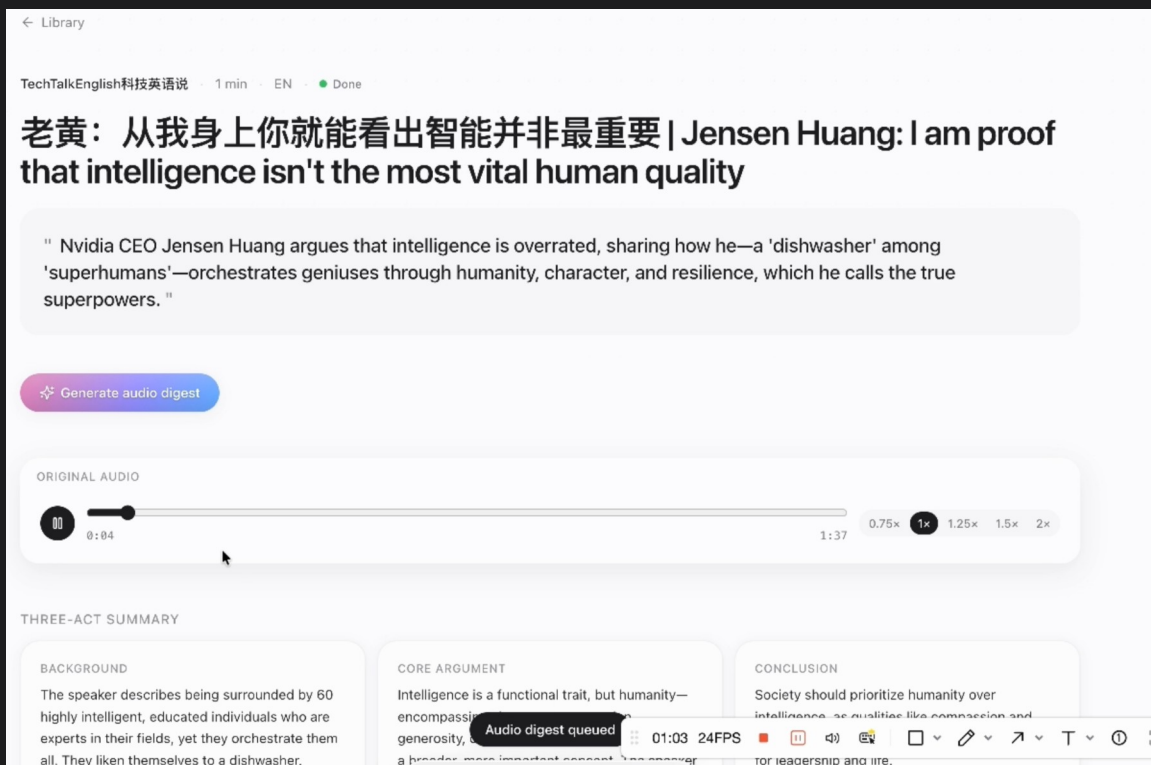


GotIt

Local end-to-end podcast summarization tool

Compress an hour of podcast into a few minutes you can actually digest

group 1 · 2026.05



The problem we want to solve

- 1 podcast episode = 40 – 90 minutes
- Want to know what it covers / is it worth listening / which parts to revisit
- Brain handles a 30-second summary far better than a 1-hour audio
- Existing tools: too heavy (Otter), no chapters (YT auto-captions), no compression (raw transcript)

ONE-LINE POSITIONING

Drop in a link. 10 minutes later you get a hook, three-act, chapters, quotes, named entities, plus a 30-second listenable TTS digest.

Demo walkthrough

01

Paste a link

YouTube / Bilibili / mp3

02

Watch live progress

WebSocket push · fetch → ASR → summary → chapters → TTS

03

Open the detail page

hook · three acts · chapter timeline · click a quote to jump back into the audio

04

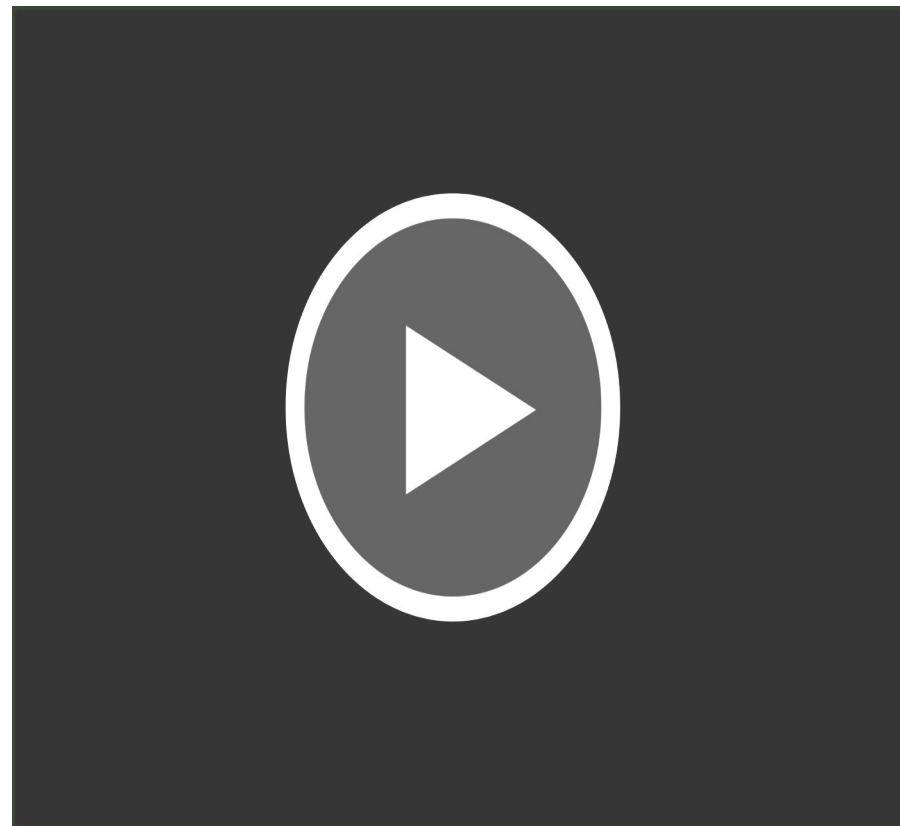
Audio overview

Doubao bigmodel TTS narrates the summary

05

One-click export

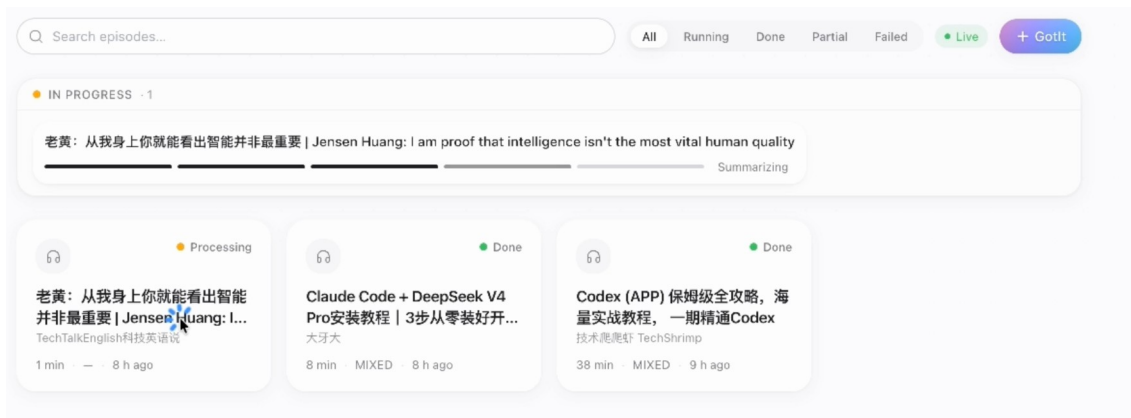
.md / .json / .mp3



The real UI: list + detail

LIST PAGE

- Apple-style restrained status dots (green / amber / red)
- Live task bar · search · 5 status filters
- Submit modal with three tabs: local / URL / YT-BB
- “+ GotIt” primary button with AI rainbow gradient



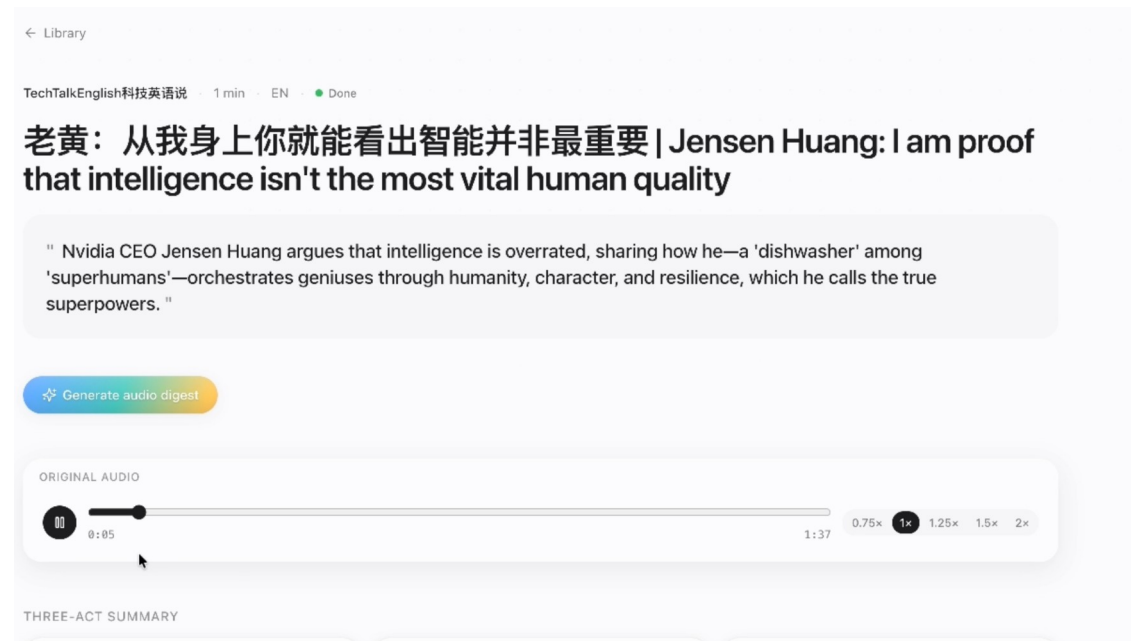
DETAIL PAGE · the most polished one

Hook quote in large type — one-line positioning under 30 chars

Sticky original audio player — pinned to the top, never disappears on scroll

Clickable chapter quotes — interpolated timestamps · seek + autoplay on click

AI Digest player — flowing rainbow button, visually distinct from the original



System architecture

FRONTEND

React 19 + Tailwind + framer-motion + Aceternity · :5174

↓ *HTTP /api* · *WS /api/ws/jobs* ↓

FASTAPI · uvicorn :8000

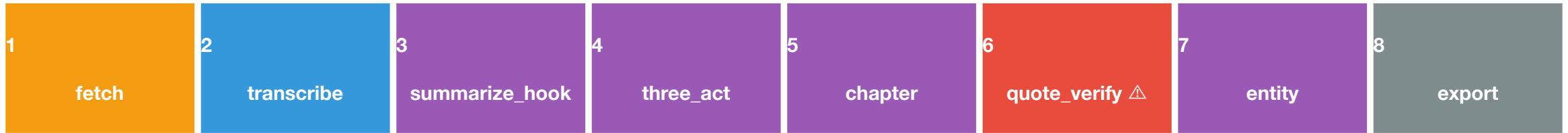
Pipeline runner (asyncio)

- 8 stages · tenacity retries · stage_progress → SQLite
- event bus → WebSocket broadcast

External services (see P8 stack)

yt-dlp / ffmpeg · Doubao ASR / TTS · DeepSeek LLM

End-to-end pipeline · 8 stages



+ tts (separate sub-flow, on-demand, never blocks the main pipeline — see P9)

Independent retry

tenacity exponential backoff · a failed stage doesn't restart the whole job

Persisted progress

stage_progress lands in SQLite · resumable across restarts

Live events

/api/ws/jobs pushes to every client · single-stage Retry button

⚠ stage 6 quote_verify is custom: NFKC + substring match + per-character interpolation for timestamps — see P10 bug #5

Stack & why we picked each piece

Layer	Choice	Why this one
Backend	FastAPI + SQLAlchemy 2 + Alembic + Pydantic 2	Strict types · async-friendly · controlled migrations
Orchestration	asyncio + tenacity	No extra queue — local tool, this is enough
Frontend v1	React 19 + Vite + plain CSS	Feature-complete · minimal deps
Frontend v2	React 19 + Tailwind + framer-motion + Aceternity	Visual A/B experiment
ASR	Doubao bigmodel 2.0 (streaming / file)	Strong on Chinese · pay-per-char pricing
LLM	DeepSeek (default) · Qwen · Anthropic	Cost-effective + multi-vendor abstraction (see P9)
TTS	Doubao v1 standard · Doubao bigmodel 2.0 (WS)	Dual track with fallback
Downloader	yt-dlp + ffmpeg	Handles YouTube and Bilibili
Testing	pytest + Vitest · domain ≥ 80%	Pure-functional domain · CI hard gate

Multi-vendor abstraction + one tricky adapter

ABSTRACTION

- ASR: doubao / openai_whisper / qwen — flip one env var
- LLM: deepseek / qwen / anthropic — same
- TTS: doubao legacy / bigmodel / qwen — same
- create_xxx_client() factory + Protocol interface · callers stay oblivious

WHAT I DID

- Ported the official demo into _doubao_big tts_protocol.py · fixed X-API-App-Key → X-API-App-Id
- thread + asyncio.run to bridge the async WS into a sync pipeline stage
- On failure, dump the first 500 chars of the server response straight into job.error

DOUBAO BIGMODEL TTS 2.0 PROTOCOL

No HTTP one-shot endpoint — only a WebSocket binary stream:

```
[4-byte header]
  version / type / flags / serialization
[4-byte event id] (optional)
[session_id length + bytes] (optional)
[4-byte payload size + payload]
```

EVENT FLOW

StartConnection → StartSession → TaskRequest → FinishSession → SessionFinished

Audio frames stream back chunked via AudioOnlyServer

“Not calling an off-the-shelf pip package — I implemented the protocol layer myself.”

Hard-won bugs I tracked down myself

#	Symptom	Root cause	Fix / lesson
1	TTS kept returning 401	Doubao expects Authorization: Bearer;<token> — semicolon, not space	1-line fix · not every API follows OAuth
2	TTS 400 'exceed max len'	v1 single text payload $\leq$ 1024 bytes	_split_text_for_tts splits on sentence boundaries by utf-8 bytes · protocol detail buried in footnote
3 ⚠	BigTTS 'concurrency exceeded' forever	resource_id was an internal numeric id; correct value is the semantic string seed-tts-2.0	Env fix · don't trust error messages at face value
4	digest.mp3 came out empty, no error	language=mixed routed to an English voice synthesizing Chinese → server silently dropped output	_voice_type maps mixed to Chinese · silent failure is worse than an exception
5 ⚠	Multiple quotes shared the same timestamp	Old logic returned the segment start — no interpolation	Rewrote verify_against_segments to interpolate by character position in segment · always look at the data, not just the schema
6	Worker crashed on retry	quote_verify re-run hit the (chapter_id, idx) unique constraint	delete_for_chapters cleans up first + unit test · idempotency is table stakes

“None of these were the kind of thing an AI could surface up front — they came from reading responses, reading the protocol, reading the DB.”

SDD: write the spec before the code

specs/001-podcast-summary/

└─ spec.md

└─ plan.md

└─ research.md

└─ data-model.md

└─ tasks.md

└─ quickstart.md

└─ ui-brief.md

└─ contracts/

└─┬─ http-api.md

└─┬─ job-events.md

└─┬─ episode-output.schema.json

└─┬─ design/...

feature spec (stories + acceptance)

implementation plan + complexity log

vendor trade-off (≥ 2 candidates each)

entity relationships

dependency-ordered tasks

getting-started guide

UI design brief

REST contract

WS event schema

UI mockups

Git log: 78 tasks

task-T078: Verify clean checkout startup.

task-T077: Polish final report zhangzuhao

task-T076: Update detail report zhangzuhao

task-T075: Add smoke test script zhangzuhao

task-T074: Add CI coverage gata...

constitution.md v1.0.0

5 non-negotiable principles

- 1

Ship in English docs / comments / commits
- 2

Python 3.11+ unified backend stack
- 3

Domain ≥ 80% coverage **NON-NEGOTIABLE**
- 4

Externalized config env / .env · no hardcoded secrets
- 5

Versioned prompts prompts/ directory, every prompt versioned

fix allow long line line nodes thurguhao

fix switch Double A&B to file recognition

fix support SPOCS process for ASR thurguhao

task-T078: Verify clean checkout startup.

task-T077: Polish final report zhangzuhao

task-T076: Update detail report zhangzuhao

task-T075: Add smoke test script zhangzuhao

task-T074: Add CI coverage gata...

task-T073: Add forward search subtasks

task-T072: Test US4 concurrency thurguhao

task-T071: Implement batch episodes API

task-T070: Add concurrent job queue thurguhao

task-T069: Add digest controls to detail

task-T068: Test digest endpoint thurguhao

task-T067: Implement digest API thurguhao

task-T066: Add optional TTS pipeline sta...

task-T065: Implement digest script build

task-T064: Implement Double TTS adapt...

task-T063: Test US2 quote jump thurguhao

task-T062: Implement quote verification

task-T061: Confirm audio range export...

task-T060: Implement entity panel thurguhao

task-T059: Implement chapter outline co...

task-T058: Extend JSON exporter thurguhao

task-T057: Extend markdown exporter thurguhao

task-T056: Extend pipeline with US2 stag...

task-T055: Test entity extractor thurguhao

task-T054: Implement entity extractor thurguhao

task-T053: Test quote verifier thurguhao

task-T052: Implement quote verifier thurguhao

task-T051: Test structured parser US2 thurguhao

task-T050: Implement chapter payload p...

task-T049: Test chapter segmenter thurguhao

task-T048: Implement chapter segmenter...

task-T047: Implement episode detail pag...

task-T046: Implement episode list pag...

task-T045: Implement frontend API client

task-T044: Test US1 resume thurguhao

task-T043: Test US1 end to end thurguhao

task-T042: Wire Websocket broadcast...

task-T041: Implement jobs API thurguhao

task-T040: Implement episodes API thurguhao

task-T039: Test JSON export scheme thurguhao

task-T038: Implement JSON exporter thurguhao

task-T037: Implement markdown exports...

task-T036: Wire US1 pipeline stages thurguhao

task-T035: Test structured parser US1 thurguhao

task-T034: Implement three-act parser thurguhao

task-T033: Implement one-act parser thurguhao

task-T032: Implement Detailed LLM ad...

task-T031: Test transcript postprocess thurguhao

task-T030: Implement transcript postpro...

task-T029: Implement Double ASR stag...

task-T028: Implement YouTube ingest thurguhao

task-T027: Implement direct URL ingest...

task-T026: Implement local file ingest thurguhao

task-T025: Wire restart recovery thurguhao

task-T024: Implement pipeline skeleton...

task-T023: Define provider starter start

task-T022: Seed versioned prompt flori...

task-T021: Test prompt assembler thurguhao

task-T020: Implement prompt assembler...

task-T019: Implement Websocket progr...

task-T018: Add structured API logging thurguhao

task-T017: Implement FastAPI user factor...

task-T016: Implement persistence respon...

task-T015: Implement persistence respon...

task-T014: Initialize Alembic migrations thurguhao

task-T013: Implement persistence model...

task-T012: Implement settings model thurguhao

task-T011: Add repository ignore rules thurguhao

task-T010: Configure pytest coverage gata...

task-T009: Seed evaluator report thurguhao

task-T008: Seed implementation detail re...

task-T007: Author RQ4MC back start thurguhao

task-T006: Add project Metadata thurguhao

task-T005: Author environment example...

task-T004: Configure frontend test thurguhao

task-T003: Configure backend test and fu...

task-T002: Initialize the frontend SPA thurguhao

task-T001: Create the backend Python thurguhao

task-T000: Create the top-level directory

THANKS

Q & A